

OTIMIZAÇÃO DE CÓDIGO POR COMPILADORES - CAP372 2024

- ▶ Livro **High Performance Computing (RISC Architectures, Optimization Benchmarks)** 2nd edition by Charles Severance, Kevin Dowd
- ▶ Objetivo: fornecer respostas corretas e executar rapidamente;
- ▶ Código gerado pode ser maior, mas com menos instruções executáveis ou com mais instruções porém mais eficiente;
- ▶ Tipos de otimização: clássicas e dependentes do h/w;
- ▶ Compilação em Linguagem Intermediária (LI);

$A = -B + C * D / E$

$T1 := D / E$

$T2 := C * T1$

$T3 := -B$

$A := T3 + T2$

Compilação para linguagem Intermediária

- ▶ instruções simples, próximas da **linguagem de máquina**;
- ▶ operador, operandos e resultado: **$X := Y \text{ op } Z$**
- ▶ referência a variáveis temporárias (uso de registradores);
- ▶ tratamento de desvios: separação do cálculo do contador e goto's para endereços absolutos;
- ▶ idéia básica: formar BLOCOS BÁSICOS (cada um com uma única entrada e uma única saída) tão independentes quanto possível entre si e substituir os desvios por um GRAFO DE FLUXO;
- ▶ identificar, dentro de cada bloco, as variáveis definidas e as utilizadas;

Exemplo laço (*loop*)

```
while (j < n) {  
  k = k + j*2;  
  m = j*2;  
  j++;  
}
```

```
A::  
  t1 := j  
  t2 := n  
  t3 := t1 < t2  
  jmp (B) t3  
  jmp (C) TRUE
```

```
B::  
  t4 := k  
  t5 := j  
  t6 := t5 * 2  
  t7 := t4 + t6  
  k := t7  
  
  t8 := j  
  t9 := t8 * 2  
  m := t9  
  t10 := j  
  t11 := t10 + 1  
  j := t11  
  jmp (A) TRUE
```

DAG - Directed Acyclic Graph (bloco B revisto)

Possibilita a análise das variáveis definidas e utilizadas.

- ▶ internamente aos blocos básicos;
- ▶ identificam as dependências entre variáveis;
- ▶ otimização blocos (menor tempo de execução e menor uso de registradores);
- ▶ “top to bottom”;

```
B::  
t4 := k  
t5 := j  
t6 := t5 * 2  
m := t6  
t7 := t4 + t6  
k := t7  
t11 := t5 + 1  
j := t11  
jmp (A) TRUE
```

Análise do fluxo de dados entre blocos

- ▶ identificar as variáveis definidas e utilizadas em cada bloco;
- ▶ identificar variáveis independentes;
- ▶ eliminar expressões redundantes;
- ▶ mover atribuições de um bloco para outro;
- ▶ possibilita liberar recursos (menor uso de registradores e menor número de instruções);

Otimização de LOOPS

- ▶ importância de otimizar;
- ▶ identificar o loop (nó dominante e o caminho de retorno);
- ▶ remover instruções “invariantes”;
- ▶ identificar trechos que utilizam o contador;
- ▶ exemplo;

Código referente à figura (grafo de fluxo)

```
.....  
NNODES = 0                                A  
DO 10 I=1,N                                B  
    J = LIST(I)                            B  
    IF (J.EQ.0) GOTO 30                     B  
20    J = NEXT(J)                           C  
    NNODES = NNODES + 1                     C  
30    IF (J.NE.0) GOTO 20                   D  
10    CONTINUE                              E
```

Geração do Código Objeto

- ▶ específico para o h/w;
- ▶ otimizações incluem a distribuição de recursos e escalonamento de instruções;
- ▶ otimizações por s/w (compilador) *versus* otimizações por h/w (processador);
- ▶ compilação para linguagem intermediária de nível 2;

- ▶ (continuação) tentar prover maior paralelismo para arquiteturas multiprocessadas ou superescalares ou VLIW;
- ▶ remoção de “dead code”;
- ▶ exemplo: RISC-6000 e processadores atuais;
 - ▶ arquitetura Harvard;
 - ▶ auto-incremento de registradores;
 - ▶ h/w pre-fetching;
 - ▶ renomeamento de registradores;
 - ▶ execução “especulativa” (branch delay slot);

Técnicas clássicas de otimização

Entender o que o compilador pode e o que não pode fazer ajuda a programar melhor.

- ▶ Propagação de cópias (**copy propagation**, intra e inter-blocos).

$X = Y$

$Z = 1. + X$

$X = Y$

$Z = 1. + Y$

- ▶ Desdobramento de constantes (**constant folding** (calcula constantes em tempo de compilação)).

INTEGER I,K

PARAMETER (I = 100)

k = 200

J = I + K

Técnicas clássicas de otimização

- ▶ Remoção de código inoperante (**dead code removal**).
 - ▶ Código inatingível ou que gera resultados que não serão empregados, gerado pelo autor do s/w ou por LI's.
 - ▶ Não há warning na otimização (somente na compilação normal);

- ▶ Redução do grau da operação (**strength reduction**).

$Y = X**2$

$J = K*2$

$Y = X*X$

$J = K + K$

- ▶ Renomeamento de variáveis (**variable renaming**), análoga ao renomeamento de registradores (register renaming), permite maior grau de paralelismo e clareza.

Técnicas clássicas de otimização

- Renomeamento de variáveis (continuação)

```
X = Y*Z;  
Q = R + X + X;  
X = A + B;
```

```
X0 = Y*Z;  
Q = R + X0 + X0;  
X = A + B;
```

- Eliminação de subexpressões comuns (**common subexpression elimination**) (dentro de uma rotina).

```
D = C * (A + B)  
E = (A + B)/2
```

```
temp = A + B  
D = C * temp  
E = temp/2
```

Também aplicável ao cálculo de endereços:

```
address(A) + (I-1)*sizeof_datatype(A) +  
+ (J-1)*sizeof_datatype(A) * column_dimension(A)
```

Técnicas clássicas de otimização

- ▶ Reposicionamento de código invariante para fora do loop (**loop invariant code motion**).

```
DO 10 I = 1,N  
    A(I) = B(I) + C*D  
    E = G(K)  
10 CONTINUE
```

```
temp = C*D  
DO I = 1,N  
    A(I) = B(I) + temp  
ENDDO  
E = G(K)
```

- ▶ Simplificação de variáveis induzidas (**induction variable simplification**).
 - ▶ variáveis induzidas são aquelas calculadas linearmente a partir do contador;
 - ▶ especialmente útil para processadores que provém incremento automático de registradores;

Técnicas clássicas de otimização

- ▶ Simplificação de variáveis induzidas (**induction variable simplification**).

```
DO I = 1,N  
  address = base_address(A)  
           + (I-1) * sizeof_datatype(A)  
ENDDO
```

```
address = base_address(A) - sizeof_datatype(A)  
DO I = 1,N  
  address = address + sizeof_datatype(A)  
ENDDO
```

- ▶ Detecção de variáveis residentes em registradores (**register variable detection**).
- ▶ análise de fluxo de dados para determinar variáveis que podem ser residentes em registradores;
- ▶ importante na arquitetura RISC (evitar load/store);

Considerações finais

- ▶ benchmarking implica no maior grau de otimização possível;
- ▶ pre-compilador pode substituir chamadas a subrotinas por inserção do código correspondente para evitar mudanças de contexto (**inlining**);
- ▶ otimizações clássicas + dependentes de h/w;
- ▶ **linker** procura agrupar conjuntamente o programa e as rotinas no módulo de carga para minimizar o cache miss de instruções;
- ▶ otimizador post-linkage para realocar o uso de registradores e reordenar os módulos do programa para melhorar a utilização do cache;
- ▶ decisões **run-time** podem se sobrepor em relação às feitas em **compile-time**;

Ex. incremento paralelismo instrução X redução cache misses

```
for (i = 0; i < 512; i = i+1)
    for (j = 1; j < 512; j = j+1)
        x[i][j] = 2 * x[i][j-1];
```

```
for (i = 0; i < 512; i = i+1)
    for (j = 1; j < 512; j = j+4) {
        x[i][j]    = 2 * x[i][j-1];
        x[i][j+1] = 2 * x[i][j];
        x[i][j+2] = 2 * x[i][j+1];
        x[i][j+3] = 2 * x[i][j+2];
    };
```

```
for (j = 1; j < 512; j = j+1)
    for (i = 0; i < 512; i = i+4) {
        x[i][j]    = 2 * x[i][j-1];
        x[i+1][j] = 2 * x[i+1][j-1];
        x[i+2][j] = 2 * x[i+2][j-1];
        x[i+3][j] = 2 * x[i+3][j-1];
    };
```

Clareza de código

- ▶ Preceitos de estilo;
- ▶ Comentários e separações;
- ▶ Escolha adequada de nomes de variáveis;
- ▶ Variáveis em FORTRAN;
- ▶ Uso de constantes (PARAMETER, # define):

```
C*****  
INTEGER NX  
REAL EPSILON  
PARAMETER (EPSILON = 1.0E-38, NX = 100);  
C*****
```

Clareza de código

- Compatibilidade nas atribuições de valores a variáveis:

```
C*****  
    double precision A  
    A = 1/3  
    A = 1./3.  
    A = 1.0d0/3.0d0
```

```
C*****
```

- Uso de COMMON (FORTRAN):

```
C*****  
    IMPLICIT NONE  
    INTEGER MAXPTS  
    PARAMETER (MAXPTS = 1000)  
    REAL X(MAXPTS), Y(MAXPTS)  
    COMMON /POINTS/ X, Y
```

```
C*****
```

- Uso de INCLUDE (C & FORTRAN);

Clareza de código

- Forma de armazenamento dos dados:

```
C*****  
PROGRAM MAIN  
INTEGER M,N  
PARAMETER (N=3, M=4)  
DOUBLE PRECISION C(N,M)  
CALL FAZ (C, N*M)  
END  
  
C*****  
SUBROUTINE FAZ(C,K)  
DOUBLE PRECISION C(K)  
DO I = 1,K  
    C(K) = 0.0D0  
ENDDO  
  
C*****
```

Portabilidade de código

- ▶ “Funcionava e deixou de funcionar...” (código fora dos padrões da linguagem);
- ▶ Aliasing:

```
C*****  
REAL A, B, C  
INTEGER I  
EQUIVALENCE (A,I)  
B = A  
CALL FOO (A,A)  
CALL BAR (A,B)  
CALL BAZ (A,I)  
C*****  
PROGRAM MAIN  
INTEGER IX = 1  
CALL ADD1 (IX,IX)  
WRITE (*,*) IX  
END
```

Portabilidade de código

- ▶ Ainda aliasing:

```
SUBROUTINE ADD1 (IA,IB)
  INTEGER IA, IB
  ...
  IF (IA.EQ.1) IB = IA +1
  IF (IB.EQ.2) IA = 1
END
C*****
```

- ▶ Incompatibilidade de tipos de variáveis (exemplo IEEE-754);
- ▶ Problemas de armazenamento devido a tipos diferentes de variáveis - EQUIVALENCE (Fortran) ou UNION (C)

```
...
union {
    int i[2];
    double x;
} foo;
...
```

- ▶ uso de ferramentas de alocação automática de memória;

Restrições de alinhamento de referências à memória

TIPO DE DADO ALINHAMENTO (múltiplo de)

```
-----  
double precision            8 byte (8)  
integer, real, logical    4 byte (4)  
integer*2                  2 byte (2)  
byte, character            1 byte (-)  
-----
```

```
REAL*4 A, B; REAL*8 R
```

```
COMMON /BAR/ A, R, B
```

```
...
```

```
PROGRAM BAR
```

```
INTEGER IARRAY (100000)
```

```
COMMON /STUFF/ IARRAY
```

```
CALL FOO (IARRAY(2),49999)
```

```
END
```

```
SUBROUTINE FOO (DPVAL,N)
```

```
DOUBLE PRECISION DPVAL(50000)
```

```
DO 10 I = 1, N
```

```
    DPVAL(I) = 1.0D0
```

```
10 CONTINUE
```

```
END
```

Restrições de alinhamento de referências à memória

```
-----  
main ()  
{  
    int *p;  
    double *q;  
    int i;  
  
    p = (int *) malloc (sizeof(int) * 3);  
    q = (double *) (p + 1);  
  
    for (i=0; i<100000; i++)  
        *q = (double) 1.0;  
}
```

(mas tipos diferentes podem conviver numa struct)

TEMPORIZAÇÃO & PROFILING

- ▶ Definição e necessidade de *timing/profiling*.
- ▶ Erros devido à frequência de amostragem.
- ▶ Temporização e profiling devem ser feitos para todas as instâncias de dados de entrada.
- ▶ Otimização: *timing/profiling* + *code tuning*;
- ▶ Técnicas e ferramentas:
 1. Comando **time** (UNIX/LINUX)
 2. Temporização de partes do código através de rotinas do sistema operacional.
 3. *Profiling*.
 4. Uso de contadores de *H/W* da CPU (clock, L1/L2 misses, flops, TLB misses, etc.) - PAPI (Performance API)

Comando “time” (UNIX/LINUX)

- ▶ **elapsed time** (tempo real transcorrido) inclui o **CPU time** gasto pelo seu programa
- ▶ **CPU time = user time + kernel time**
- ▶ **kernel/system time** deve-se a chamadas do seu programa a rotinas do sistema operacional (inclui page faults, referências desalinhadas à memória, tratamento de exceções, etc.).
- ▶ quando **elapsed time** >>> **cpu time**, tem-se alto grau de multiprogramação (por ex. muitos processos de usuários), muito **input/output**, relativo a tempo de acesso ao disco rígido e outros periféricos (terminais, dispositivos de fitas, etc.).

PROFILING

- ▶ Perfis de execução e a Lei de Amdahl.
- ▶ Ferramentas mais comuns (UNIX/LINUX): **prof** e **gprof**
- ▶ Uso do **prof**:

```
% cc loops.c -p -o loops  
% ./loops  
% prof loops
```

- ▶ Uso do **gprof**:

```
% cc loops.c -pg -o loops  
% ./loops  
% gprof loops
```


Exemplo - prof

```
#include <math.h>

main () {
    int i;
    for (i=0;i<100;i++) {
        if (i == 2*(i/2)) loop1 ();
        loop2 ();
        loop3 ();
        loop4 ();
    }
    loop1 () {
        int i;
        double x[10000];
        for (i=0;i<10000;i++)
            x[i] = x[i]*x[i];
    }
    loop2 () {
        int i;
        double x[50000];
        for (i=0;i<50000;i++)
            x[i] = x[i]*x[i];
    }
}
```

Exemplo - prof (continuação)

```
loop3 () {  
    int i;  
    double x[100000];  
    for (i=0;i<100000;i++)  
        x[i] = x[i]*x[i]; }  
loop4 () {  
    int i;  
    double x[100000];  
    for (i=0;i<100000;i++)  
        x[i] = pow(x[i],3.5); }
```

```
*****  
Name      %Time  Seconds  Cumsecs  #Calls   msec/call  
.pow      47.6    30.53    30.53    10000000  0.0031  
._mcount  21.4     13.71    44.24  
.loop4    12.0     7.69     51.93    100       76.90  
.loop3    9.4      6.06     57.99    100       60.60  
.loop2    4.7      3.02     61.01    100       30.20  
.loop1    0.4      0.28     63.95    50        5.6  
...  
.main     0.0      0.00     64.18    1         0.
```

Exemplo - gprof

index	%time	self	descendents	called/total called+self called/total	parents name children	index
		0.00	44.45	1/1	._start	[2]
[1]	53.0	0.00	44.45	1	.main	[1]
		6.03	37.65	100/100	.loop3	[3]
		0.50	0.00	100/100	.loop2	[10]
		0.27	0.00	50/50	.loop1	[17]

		6.03	37.65	100/100	.main	[1]
[3]	52.1	6.03	37.65	100	.loop3	[3]
		7.13	30.52	100/100	.loop4	[4]

		7.13	30.52	100/100	.loop3	[3]
[4]	44.9	7.13	30.52	100	.loop4	[4]
		30.52	0.00	10000000/10000000	.pow	[6]

		30.52	0.00	10000000/10000000	.loop4	[4]
[6]	36.4	30.52	0.00	10000000	.pow	[6]
		0.00	0.00	100/100	.loginner2	[20]
		0.00	0.00	100/100	.expinner2	[19]

Exemplo - gprof (continuação)

```
-----  
                0.50      0.00      100/100      .main [1]  
[10]      0.6      0.50      0.00      100      .loop2 [10]  
-----
```

```
                0.27      0.00      50/50      .main [1]  
[17]      0.3      0.27      0.00      50      .loop1 [17]  
-----
```

```
% %   cumulative   self          self   total  
% time  seconds  seconds    calls  ms/call  ms/call  name  
% 41.5    34.85    34.85             0.00    0.00  .__mcount [5]  
% 36.4    65.37    30.52 100000000    0.00    0.00  .pow [6]  
%  8.5    72.50     7.13     100    71.30   376.50  .loop4 [4]  
%  7.2    78.53     6.03     100    60.30   436.80  .loop3 [3]  
%  0.6    81.37     0.50     100     5.00     5.00  .loop2 [10]  
%  0.3    83.65     0.27     50     5.40     5.40  .loop1 [17]  
%  0.0    83.91     0.00      1     0.00  44450.00  .main [1]
```

Conceitos de paralelismo para CPU/núcleo

- ▶ Otimizações possíveis: aumentar o grau de paralelismo, melhorar acesso à memória, utilizar algoritmos mais eficientes.
- ▶ Grau de paralelismo: para máquinas monoprocessadas, relaciona-se com o número de instruções que podem ser executadas concorrentemente (num pipeline) ou simultaneamente (em unidades funcionais diferentes) pela CPU. Fala-se em paralelismo em nível de instrução, ou em granulação fina.
 - ▶ Aumentar o paralelismo significa eliminar dependências de dados e de instruções.
 - ▶ “Ajudar” o compilador a reconhecer código paralelizável, deixando mais claras as não-dependências.
 - ▶ Não há receitas prontas.
 - ▶ Técnicas diferentes para processadores escalares e vetoriais;

Conceitos de paralelismo para CPU/núcleo (continuação)

- ▶ Arquitetura RISC: paralelismo graças a pipelines.
- ▶ Aritmética vetorial: paralelismo graças a operações com vetores/matrizes.

```
DO I = 1, N
```

```
    A(I) = B(I) * C
```

```
ENDDO
```

```
A ← A + B * C
```

- ▶ Supercomputação: processadores vetoriais *versus* máquinas paralelas.
- ▶ Programar evitando dependências de dados e de controle, inclusive para poder portar o código para máquina paralela.

DEPENDÊNCIA DE DADOS

```
DO 10 I = 1,N  
  A(I) = A(I) + B(I)  
10 CONTINUE  
  A(I)   = A(I)   + B(I)  
  A(I+1) = A(I+1) + B(I+1)  
  A(I+2) = A(I+2) + B(I+2)
```

D1) DEPENDÊNCIAS DE FLUXO

```
DO 10 I = 2,N  
  A(I) = A(I-1) + B(I)  
10 CONTINUE  
  
  A(I)   = A(I-1) + B(I)  
  A(I+1) = A(I)   + B(I+1)  
  A(I+2) = A(I+1) + B(I+2)  
  
DO 10 I = 2,N,2  
  A(I)   = A(I-1) + B(I)  
  A(I+1) = A(I-1) + B(I) + B(I+1)  
10 CONTINUE
```

D2) ANTI-DEPENDÊNCIAS

X = A/B

Y = X + 2.0

X = D - E

temp = A/B

Y = temp + 2.0

X = D - E

DO 10 I = 1,N

A(I) = B(I)*E

B(I) = A(I+2)*C

10 CONTINUE

VECTOR PROCESSOR:

A(I) = B(I)*E

A(I+1) = B(I+1)*E

A(I+2) = B(I+2)*E

...

B(I) = A(I+2)*C (erro !!!)

B(I+1) = A(I+3)*C

B(I+2) = A(I+4)*C

(seria necessário utilizar variáveis temporárias para vetorizar)

RISC PROCESSOR:

```
A(I)    = B(I)*E
B(I)    = A(I+2)*C
A(I+1)  = B(I+1)*E
B(I+1)  = A(I+3)*C
A(I+2)  = B(I+2)*E
B(I+2)  = A(I+4)*C
```

D3) DEPENDÊNCIAS DE SAÍDA *output dependencies*)

```
DO 10 I = 1,N
  A(I)    = C(I)*2.
  A(I+2)  = D(I) + E
10 CONTINUE
```

```
A(I)    = C(I)*2.
A(I+1)  = C(I+1)*2.
A(I+2)  = C(I+2)*2.
A(I+2)  = D(I) + E      (erro !!!)
A(I+3)  = D(I+1) + E
A(I+4)  = D(I+2) + E
```

Distância de Dependência:

< 0 dependência de fluxo
= 0 não-dependência
> 0 anti-dependência

```
DO 10 I = 1,N  
    A(I) = B(I)*E  
    B(I) = A(I+2)*C  
10 CONTINUE
```

“Referências Ambíguas”

```
DO 10 I = 1,N  
    A(I) = B(I)*E  
    B(I) = A(I+K)*C    (K desconhecido)  
10 CONTINUE
```

DEPENDÊNCIAS DE CONTROLE

- ▶ Reposicionamento de instruções “custosas” (exemplo).
- ▶ *Branch delay slot.*

Otimização de laços - “loop unrolling”

```
DO I=1,N  
    A(I) = A(I) + B(I) * C  
ENDDO
```

(incluir loop pré-condicionador)

```
II = IMOD(N,4)  
DO I = 1, II  
    A(I) = A(I) + B(I) * C  
ENDDO
```

```
DO I = 1+II,N,4  
    A(I)    = A(I)    + B(I)    * C  
    A(I+1)  = A(I+1)  + B(I+1)  * C  
    A(I+2)  = A(I+2)  + B(I+2)  * C  
    A(I+3)  = A(I+3)  + B(I+3)  * C  
ENDDO
```

“Loop unrolling” - quando não é conveniente

- ▶ poucas iterações (contador baixo);
- ▶ loops com muitas instruções (*fat loops*);
- ▶ loops com chamadas de subrotinas;

```
DO I = 1, N  
    CALL SHORT (A(I), B(I), C(I))  
ENDDO
```

...

```
SUBROUTINE SHORT (A,B,C)  
A = A + B * C  
RETURN  
END
```

```
DO I = 1+II, N, 4  
    A(I)    = A(I) + B(I) * C  
    A(I+1)  = A(I+1) + B(I+1) * C  
    A(I+2)  = A(I+2) + B(I+2) * C  
ENDDO
```

- ▶ “Unrolling” possível com **inlining** de rotinas curtas

“Loop unrolling” - laços com desvios, quando CPU tem H/W específico para desvios

```
DO I = 1, N
  DO J = 1, N
    IF (B(J,I).GT.1.0)
      A(J,I) = A(J,I) + B(J,I)*C
    ENDDO
  ENDDO
```

```
DO I = 1, N
  DO J = II+1, N, 4
    IF (B(J,I).GT.1.0)
      A(J,I) = A(J,I) + B(J,I)*C
    IF (B(J+1,I).GT.1.0)
      A(J+1,I) = A(J+1,I) + B(J+1,I)*C
    IF (B(J+2,I).GT.1.0)
      A(J+2,I) = A(J+2,I) + B(J+2,I)*C
    IF (B(J+3,I).GT.1.0)
      A(J+3,I) = A(J+3,I) + B(J+3,I)*C
    ENDDO
  ENDDO
```

“Loop unrolling” - laços recursivos (depend. fluxo) e implica em unrolling às custas de mais operações

```
DO I = 2, N  
  A(I) = A(I) + A(I-1)*B  
ENDDO
```

```
A(I)    = A(I) + A(I-1)*B  
A(I+1)  = A(I+1) + A(I)*B  
A(I+2)  = A(I+2) + A(I+1)*B  
A(I+3)  = A(I+3) + A(I+2)*B
```

```
DO I = 2, N, 2  
  A(I+1) = A(I+1) + (A(I) + A(I-1)*B)*B  
  A(I)   = A(I)   + A(I-1)*B  
ENDDO
```

“Loop unrolling” - problemas em potencial

- ▶ Compilador “decide” se faz ou não unrolling;
- ▶ Escolha do fator de “unrolling” (tipicamente 3 ou 4)
- ▶ *Register thrashing* (escassez de registradores);
- ▶ *Instruction cache miss* (instruções em excesso);
- ▶ limitações de *memory bandwidth* em máquinas multiprocessadas;

Loop “unrolling” - laços externos:

```
/* loop triplo original */
for (i=0; i<n; i++)
    for (j=0; j<n; j++)
        for (k=0; k<n; k++)
            a[i][j][k] = a[i][j][k] + b[i][j][k]

/* unrolling do loop intermediário */
for (i=0; i<n; i++)
    for (j=0; j<n; j+=2)
        for (k=0; k<n; k++)
            a[i][j][k] = a[i][j][k]      + b[i][j][k]
            a[i][j+1][k] = a[i][j+1][k] + b[i][j+1][k]

/* unrolling dos loops intermediário e interno */
for (i=0; i<n; i++)
    for (j=0; j<n; j+=2)
        for (k=0; k<n; k+=2)
            a[i][j][k]      = a[i][j][k]      + b[i][j][k]
            a[i][j+1][k]    = a[i][j+1][k]    + b[i][j+1][k]
            a[i][j][k+1]    = a[i][j][k+1]    + b[i][j][k+1]
            a[i][j+1][k+1]  = a[i][j+1][k+1]  + b[i][j+1][k+1]
```


Ex. Unrolling do laço externo, iterações $N \gg M$

```
DO I = 1, N
  DO J = 1, M
    A(J,I) = B(J,I) + C(J,I)*D
  ENDDO
ENDDO
```

```
II = IMOD(N,4)
DO I = 1, II
  DO J = 1, M
    A(J,I) = B(J,I) + C(J,I)*D
  ENDDO
ENDDO
```

```
DO 10 I = II+1, N, 4
  DO 20 J = 1, M
    A(J,I) = B(J,I) + C(J,I)*D
    A(J,I+1) = B(J,I+1) + C(J,I+1)*D
    A(J,I+2) = B(J,I+2) + C(J,I+2)*D
    A(J,I+3) = B(J,I+3) + C(J,I+3)*D
  ENDDO
ENDDO
```

Ex. Unrolling do laço externo, devido à dependência de dados entre iterações do loop interno

```
DO J = 1, M
  DO I = 2, N
    A(I,J) = A(I,J) + A(I-1,J)*D
  ENDDO
ENDDO
```

```
JJ = IMOD(M,4)
DO J = 1, JJ
  DO I = 2, N
    A(I,J) = A(I,J) + A(I-1,J)*D
  ENDDO
```

```
DO J = 1+JJ, M, 4
  DO I = 2, N
    A(I,J)    = A(I,J)    + A(I-1,J)*D
    A(I,J+1)  = A(I,J+1)  + A(I-1,J+1)*D
    A(I,J+2)  = A(I,J+2)  + A(I-1,J+2)*D
    A(I,J+3)  = A(I,J+3)  + A(I-1,J+3)*D
  ENDDO
ENDDO
```

Transformações Associativas

$(X + Y) + Z$ equivale a $(Y + X) + Z$ OK

Mas, $(Y + Z) + X$???

Considere: $X = Y = .00005$ e $Z = 1.0000$

► Reduções (por ex. produto escalar):

```
SUM = 0.0
```

```
DO I = 1, N
```

```
    SUM = SUM + A(I) * B(I)
```

```
ENDDO
```

C Loop unrolling para RISC

C RISC's suportam $r1*r2 + r3 \rightarrow r4$

```
SUM0 = 0.0; SUM1 = 0.0;
```

```
SUM2 = 0.0; SUM3 = 0.0
```

```
DO I = 1, N, 4
```

```
    SUM0 = SUM0 + A(I) * B(I)
```

```
    SUM1 = SUM1 + A(I+1) * B(I+1)
```

```
    SUM2 = SUM2 + A(I+2) * B(I+2)
```

```
    SUM3 = SUM3 + A(I+3) * B(I+3)
```

```
    SUM3 = SUM3 + A(I+3) * B(I+3)
```

```
ENDDO
```

```
SUM = SUM0 + SUM1 + SUM2 + SUM3
```

Loop unfolding para proc. vetorial

```
DO I = 1, N
    TEMP(I) = A(I) * B(I)
ENDDO
```

```
DO I = 1, N/2
    TEMP(I) = TEMP(I) + TEMP(I+N/2)
ENDDO
```

```
DO I = 1, N/4
    TEMP(I) = TEMP(I) + TEMP(I+N/4)
ENDDO
```

```
DO I = 1, N/8
    TEMP(I) = TEMP(I) + TEMP(I+N/8)
ENDDO
```

```
X = 0
DO I = 1, N/8
    X = X + TEMP(I)
ENDDO
```

Daxpy's: (2 flops & 3 load/stores, não conveniente para RISC's)

```
DO 10 I = 1, N
10    DY(I) = DY(I) + DA * DX(I)
```

Multiplicação de matrizes

C Inicializar com zero os elementos $C(J,I)$

```
DO K = 1, N
  DO J = 1, N
    DO I = 1, N
       $C(I,J) = C(I,J) + A(I,K)*B(K,J)$ 
    ENDDO
  ENDDO
ENDDO
```

C PASSO I: Permutar os loops mais externos

C Inicializar com zero os elementos $C(J,I)$

```
DO J = 1, N
  DO I = 1, N
    DO K = 1, N
       $C(I,J) = C(I,J) + A(I,K)*B(K,J)$ 
    ENDDO
  ENDDO
ENDDO
```

Multiplicação de matrizes

PASSO II: Loop unrolling

C Inicializar com zero os elementos C(J,I)

DO J = 1, N

DO I = 1, N

KK = IMOD (N,4)

DO K = 1, KK

C(I,J) = C(I,J) + A(I,K)*B(K,J)

ENDDO

TEMPO = 0.0

TEMP1 = 0.0

TEMP2 = 0.0

TEMP3 = 0.0

DO K = 1+KK, N, 4

TEMPO = TEMPO + A(I,K)*B(K,J)

TEMP1 = TEMP1 + A(I,K+1)*B(K+1,J)

TEMP2 = TEMP2 + A(I,K+2)*B(K+2,J)

TEMP3 = TEMP3 + A(I,K+3)*B(K+3,J)

ENDDO

C(I,J) = C(I,J)+TEMPO+TEMP1+TEMP2+TEMP3

ENDDO

ENDDO

Permutação de loops e dependência de dados

C Seria conveniente o loop em K mais externo
C para permitir unrolling dos mais internos.

```
PARAMETER (IDIM=1000,JDIM=1000,KDIM=4)
```

```
DO I = 1, IDIM
  DO J = 1, JDIM
    DO K = 1, KDIM
      D(K,J,I) = D(K,J,I) + V(K,J,I)*DT
    ENDDO
  ENDDO
ENDDO
```

C Exemplo de possíveis dependências de dados:

```
DO I = 1, N-1
  DO J = 2, N
    A(I,J) = A(I+1,J-1) * B(I,J)
    C(I,J) = B(J,I)
  ENDDO
ENDDO
```

{\bf Balanço de load/stores e flops}:

Finalizando: otimização de loops

- ▶ tornar loops vetorizáveis (eliminar desvios, inlining de rotinas curtas, permutar loops, etc.);
- ▶ aumentar grau de paralelismo (loop unrolling, rearranjo de reduções, etc.);

Eliminando a “bagunça”:

- ▶ evitar chamadas a subrotinas/procedures (pior com variáveis comuns - COMMON, EXTERNAL, etc.);
- ▶ Uso de macros (pode requerer *pre-processor*:

```
#define average(x,y) ((x+y)/2)
```

- ▶ *inlining*:
 - ▶ lista na linha de comando do compilador
 - ▶ “diretivas ” no programa fonte
 - ▶ automático

- ▶ evitar desvios/testes dentro de loops;
- ▶ evitar/otimizar testes longos:

```
if(((~i&j) && a<=0. && m== -1) || k==0)
```

```
if(k==0 || ((~i&j) && a<=0. && m== -1))
```

Eliminando a “bagunça”

- ▶ Evitar testes redundantes:

```
if (c=='A') A = TRUE;  
if (c=='B') B = TRUE;  
if (c=='C') C = TRUE;
```

```
switch (c) {  
  case 'A': A = TRUE; break;  
  case 'B': B = TRUE; break;  
  case 'C': C = TRUE; break;  
}
```

- ▶ Testes/desvios dentro de loops

```
C Não valeria a pena:  
PARAMETER (SMALL = 1.E-20)  
DO I = 1, N  
  IF (ABS(A(I)).GE.SMALL) THEN  
    B(I) = B(I) + A(I)*C  
  ENDIF  
ENDDO
```

Eliminando... ainda testes/desvios dentro de loops

- ▶ Outro exemplo: código ineficiente

```
DO I = 1, N
  IF(A(I).LT.0.) A(I) = -A(I)
ENDDO
```

C O mesmo código otimizado:

```
#define ABS(Q) (Q < 0.? -Q : Q)
...
for (i=0; i<n; i++)
  a[i] = ABS(a[i]);
```

C Cuidado: C fabs() gera chamada...

Eliminando... ainda testes/desvios dentro de loops

- ▶ Ex. de teste invariante dentro do loop:

```
DO I = 1, K
  IF (N.EQ.0) THEN
    A(I) = A(I) + B(I) *C
  ELSE
    A(I) = 0.
  ENDIF
ENDDO
```

- ▶ Ex. de teste invariante fora do loop:

```
IF (N.EQ.0) THEN
  DO I = 1, K
    A(I) = A(I) + B(I) *C
  ENDDO
ELSE
  DO I = 1, K
    A(I) = 0.
  ENDDO
ENDIF
```

Eliminando... - ainda testes/desvios dentro de loops

- ▶ Testes dependentes do contador podem ser trocados por novos loops:

```
DO I = 1, N
  DO J = 1, N
    IF (J.LT.I) THEN
      A(J,I) = A(J,I)*C
    ELSE
      A(J,I) = 0.0
    ENDIF
  ENDDO
ENDDO
```

C Note que o loop em J com o IF-THEN-ELSE foi trocado:

```
DO I = 1, N
  DO J = 1, I-1
    A(J,I) = A(J,I)*C
  ENDDO
  DO J = I,N
    A(J,I) = 0.0
  ENDDO
ENDDO
```

Eliminando... - ainda testes/desvios dentro de loops

- ▶ Testes sem dep. dados entre iterações:

```
DO I = 1, N
  DO J = 1, N
    IF (B(J).GT.1.0) THEN
      A(J,I) = A(J,I) + B(J,I)*C
    ENDIF
  ENDDO
ENDDO
```

C Pode-se fazer o loop unrolling:

```
DO I = 1, N
  DO J = 1, N, 2
    IF (B(J).GT.1.0) THEN
      A(J,I) = A(J,I) + B(J,I)*C
    ENDIF
    IF (B(J+1).GT.1.0) THEN
      A(J+1,I) = A(J+1,I) + B(J+1,I)*C
    ENDIF
  ENDDO
ENDDO
```

Eliminando... - ainda testes/desvios dentro de loops

- ▶ testes com dep. dados entre iterações:

```
C DEVEM SER EVITADOS!
```

```
DO I = 1, N
```

```
  IF (X.LT.A(I)) X = X + B(I)*2.
```

```
ENDDO
```

- ▶ Exemplo com unrolling de uma redução:

```
for (i=0; i<n; i++)
```

```
  z = a[i] > z ? a[i] : z;
```

```
z0 = 0.;
```

```
z1 = 0.;
```

```
for (i=0; i<n-1; i+=2) {
```

```
  z0 = a[i] > z0 ? a[i] : z0;
```

```
  z1 = a[i+1] > z1 ? a[i+1] : z1;
```

```
}
```

```
z = z0 < z1 ? z1 : z0;
```

Eliminando... - ainda testes/desvios dentro de loops

- Conversões de tipos de dados:

```
C "Promoção" de B de REAL*4 para REAL*8 consome tempo
REAL*8 A(20)
REAL*4 B(20)
DO I = 1, 20
    A(I) = A(I) + B(I)
ENDDO
```

- Eliminação manual de expressões comuns:

```
/* nem tudo é óbvio para o compilador: */
d = a + b + c;
e = c + b + a;
temp = a + b + c;   ???
```

```
/* chamadas a rotinas nunca são substituídas automaticamente: */
x = r*sin(a)*cos(b);
y = r*sin(a)*sin(b);
```


Eliminando... - ainda testes/desvios dentro de loops

- ▶ Ex. para recuperar último caracter de um string:

```
while (*p != ' '){
    c = *p++;
```

```
while (*p++ != ' ');
c = *--p;
```

- ▶ Evitando load/store's inúteis, através do uso de variáveis temporárias:

```
/* bom para RISC, ruim para proc. vetorial */
```

```
for (i=0; i<n; i++) {
    xold[i] = x[i];
    x[i] = x[i] + xinc[i];
}
```

```
for (i=0; i<n; i++) {
    temp = x[i];
    xold[i] = temp;
    x[i] = temp + xinc[i];
}
```

Otimização de acesso à memória

Um exemplo de “blocking” - acesso por linhas (em C) ou por colunas (FORTRAN, como abaixo):

```
DO I = 1, N
  DO J = 1, N
    A(J,I) = A(J,I) + B(I,J)
  ENDDO
ENDDO
```

C “blocking” - unrolling conjunto:

```
DO I = 1, N, 2
  DO J = 1, N, 2
    A(J,I)      = A(J,I)      + B(I,J)
    A(J+1,I)    = A(J+1,I)    + B(I,J+1)
    A(J,I+1)    = A(J,I+1)    + B(I+1,J)
    A(J+1,I+1)  = A(J+1,I+1)  + B(I+1,J+1)
  ENDDO
ENDDO
```

Otimização de acesso à memória:

C otimizando o uso da TLB:

```
DO I = 1, N, 2
  DO J = 1, N/2, 2
    A(J,I)      = A(J,I)      + B(I,J)
    A(J+1,I)    = A(J+1,I)    + B(I,J+1)
    A(J,I+1)    = A(J,I+1)    + B(I+1,J)
    A(J+1,I+1)  = A(J+1,I+1)  + B(I+1,J+1)
  ENDDO
ENDDO
DO I = 1, N, 2
  DO J = N/2+1, N, 2
    A(J,I)      = A(J,I)      + B(I,J)
    A(J+1,I)    = A(J+1,I)    + B(I,J+1)
    A(J,I+1)    = A(J,I+1)    + B(I+1,J)
    A(J+1,I+1)  = A(J+1,I+1)  + B(I+1,J+1)
  ENDDO
ENDDO
```

"Blocking" para matrizes muito grandes

```
II = MOD (N,16); JJ = MOD (N,4);  
DO I = 1, N  
    DO J = 1, JJ  
        A(J,I) = A(J,I) + B(I,J)  
    ENDDO  
ENDDO  
DO I = 1, II  
    DO J = JJ+1, N  
        A(J,I) = A(J,I) + B(I,J)  
    ENDDO  
ENDDO  
  
DO I = II+1, N, 16  
    DO J = JJ+1, N, 4  
        DO K = I, I+15  
            A(J,K) = A(J,K) + B(K,J)  
            A(J+1,K) = A(J+1,K) + B(K,J+1)  
            A(J+2,K) = A(J+2,K) + B(K,J+2)  
            A(J+3,K) = A(J+3,K) + B(K,J+3)  
        ENDDO  
    ENDDO  
ENDDO
```

Referências ambíguas/indiretas à memória

- ▶ Índices ambíguos de vetores ou matrizes dentro de loops: $A(I + K)$;
- ▶ Referências indiretas: $A(K(I))$;
- ▶ Indefinições na alocação de memória:

```
/* alocação bem definida: */  
double a[N][N], c[N][N], d;  
...  
for (i=0; i<N; i++)  
    for (j=0; j<N; j++)  
        a[i][j] = a[i][j] + c[j][i]*d;  
/* possibilidade de aliasing (a & c) */  
double *a[n], *c[N], d;  
...  
for (i=0; i<N; i++) {  
    a[i] = (double *) malloc (N*sizeof(double));  
    c[i] = (double *) malloc (N*sizeof(double));  
}  
...  
for (i=0; i<N; i++)  
    for (j=0; j<N; j++)  
        a[i][j] = a[i][j] + c[j][i]*d;
```

Referências ambíguas/indiretas à memória

```
/* idem em chamada a rotina */
main ()
{
    double a[N][N], c[N][N], d;
    ...
    sub (a,c,d,N)
}
sub (a,c,d,N)
double *a, *c, d;
int n;
{
    ...
}
```

- Medindo o tamanho do programa:

```
% size prog
text    data    bss      dec      hex      filename
4548    172     4         4724    1274     prog
```

- Medindo a atividade de *paging*

```
% vmstat 5
procs      memory                page
r  b  w    swap  free  re  mf  pi  po ...
```

Linguagens/extensões para PAD

- ▶ Bibliotecas de subrotinas matemáticas (operações com vetores/matrizes, etc.) ou específicas para outro fim:
 - ▶ proprietárias, de domínio público ou de terceiros;
 - ▶ exemplos: NAG, IMSL, BLAS, LAPACK, etc.
- ▶ Pré-processadores vetorizantes, paralelizantes e compiladores com otimização avançada, com diretivas específicas para não-dependências, permutações, não-vetorizar, não-equivalências, probabilidade de desvio, contagem de iterações, “unrolling”, não fazer transformações associativas, inlining, níveis de otimização, etc.
- ▶ Linguagem explicitamente paralelas *versus* extensões de linguagens para paralelismo;

Fortran 90

- ▶ Operações com vetores/matrizes, alocação dinâmica de memória, ponteiros, novos tipos de dados, estruturas, funções intrínsecas para vetores e matrizes, novas estruturas de controle;

- ▶ Operações com vetores e matrizes:

```
DO I = 1, N
```

```
    A(I) = A(I) + B(I)
```

```
ENDDO
```

```
DO J = 1, M
```

```
    DO I = 1, N
```

```
        A(I,J) = A(I,J) + B(I,J)
```

```
    ENDO
```

```
ENDDO
```

- ▶ Soma: **A = A + B**
- ▶ Multiplicação elemento a elemento
A = A * B
- ▶ Multiplicação de matrizes: MATMUL

F90 ainda...

- ▶ Transposta: TRANSPOSE
- ▶ Importância do *shape conformance*
- ▶ Triplets (definem seções de *arrays*)

```
REAL X(10,10), Y(100), Z(5,5,5)
```

```
...
```

```
X (10, 1:10) = Y (91:100)
```

```
X (10, :)    = Y (91:100)
```

```
Z (10:1:-2,5) = Y (1:5)
```

- ▶ funções Intrínsecas:

```
REAL A(100,10,2)
```

```
REAL B(10,10,100)
```

```
A = SIN(A)
```

```
B(:, :, 1) = COS (A(1:100:10, :, 1))
```

C Novas funções intrínsecas:

MAXVAL, MINVAL, DOT_PRODUCT, RESHAPE,

SPREAD, MERGE, SHAPE, SIZE, LBOUND,

UBOUND, ANY, ALL

Fortran 90

- ▶ novas estruturas de controle:

C estrutura DO-WHILE;

C estrutura SELECT-CASE;

C estrutura WHERE-ELSEWHERE-ENDWHERE:

```
REAL A(4,4), B(4,4), C(4,4)
```

```
...
```

```
WHERE (B.EQ.C)
```

```
    C = B + 1.0
```

```
ELSEWHERE
```

```
    A = -1.0
```

```
ENDWHERE
```

C DO-ENDDO, CYCLE & EXIT:

```
    I = 0
```

```
LOOP: DO
```

```
    I = I + 1
```

```
    IF (I.EQ.2*(I/2)) CYCLE
```

```
    IF (I.GT.10) EXIT
```

```
    WRITE(*,*) I
```

```
ENDDO LOOP
```

Linguagens/extensões para programação paralela

- ▶ High Performance FORTRAN (HPF): extensão de paralelismo para F90 (comandos extras para distribuir e sincronizar tarefas entre núcleos/CPU's);
- ▶ Bibliotecas de troca de mensagens: MPI (grande padrão atual) e PVM.
- ▶ Programação com *threads* e OpenMP (somente para *shared memory*)
- ▶ Ambientes de programação ou bibliotecas com suporte a PAD (Python, etc.)
- ▶ LLVM *Lawrence Livermore National Laboratory Virtual Machine*: conjunto de bibliotecas e compiladores que gera código executável para diversas arquiteturas

Benchmarking, Benchmarks

- ▶ *Benchmarks* são programas que medem desempenho de CPU, nuvem, IO, etc. (levam em conta execução de instruções e acesso à memória)
- ▶ **VAX MIPS**: baseado no DEC VAX 11/780 (470 k instruções inteiras /s);
- ▶ **Mflop/s** para ponto flutuante (aprox. $\text{Mflop} = \text{clock} * 2$)
- ▶ **Linpack**: eliminação gaussiana usada no Top500 (www.top500.org)
- ▶ **SPEC benchmarks - Standard Performance Evaluation Corporation**
 - ▶ SPEC tem benchmarks “para tudo” (www.spec.org) !!!
 - ▶ **SPEC CPU 2017** usa média geométrica dos tempos de execução de programas com precisão simples, precisão dupla e inteiros (ex. treinador de redes neurais, *compress* do UNIX, método de Monte Carlo, interpretador de LISP, ray tracing, compilador C da GNU, simulações em química quântica, diferenças finitas para previsão de tempo, equações de Maxwell, planilha de cálculo, etc.

Faça seu próprio benchmark

- ▶ Escolha criteriosa de um conjunto de programa que seja representativo;
- ▶ *Run time* adequado: não muito curto ou longo;
- ▶ Comparação da máquina candidata com a atual considerando tempos, MFlop/s, etc.;
- ▶ Tamanhos diferentes do problema podem levar a testar o desempenho da CPU ou do sistema de memória;
- ▶ Elaboração de *kernel* do software utilizado para benchmarking;
- ▶ cuidados com software de terceiros;

Faça seu próprio benchmark

- ▶ Tipos de benchmarking:
 - ▶ **“fluxo simples”**: conjunto de programas executados sequencialmente, *elapsed time* total e parcial, este normalizado e ponderado (exemplo da figura);
 - ▶ **“throughput”**: vários programas executados repetida e concorrentemente (A-A-A-A-... com B-B-B-B-... com C-C-C-C-...);
 - ▶ **interativos**: uso de emulador de terminal remoto ou não; outros exemplos: processadores de texto, editores, etc.
- ▶ Preparando o código para o benchmarking:
 - ▶ portabilidade: mínimo de modificações e análise da diferença numérica nos resultados;
 - ▶ evitar paging/swapping (verificar *elapsed versus CPU time*);
 - ▶ opção mais adequada de otimização de código por compilador;
 - ▶ construindo um “benchmark kit” (Makefile ou script, fontes, dados, informação);
 - ▶ cuidado com resultados fabulosos do fabricante para seu benchmark;